

Contents

Resolución Práctica 6: Clase 03	1
Escenario	1
Parte A: Consultas que debe soportar el event log	1
Parte B: Diseño orientado a consultas	2
Parte C: CQL conceptual	4
Parte D: Análisis de particiones	5
Parte E: Consistencia configurable	6
Parte F: Comparación con MongoDB	6

Resolución Práctica 6: Clase 03

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

Escenario

Se modela el event log de FlowOps con enfoque Cassandra. Cada instancia de proceso genera eventos de auditoría append-only (inicio, carga de formulario, validación, creación de tarea, aprobación o rechazo, error técnico, reintento, notificación, finalización). El event log crece mucho más rápido que las definiciones de proceso o la configuración de tenants, por lo que se diseña desde el modelo columnar, las claves de consulta y la escritura masiva.

Evento de referencia para el proceso "Alta de proveedor":

```
{
  "tenant_id": "empresa_demo",
  "process_id": "alta_proveedor",
  "instance_id": "inst_000123",
  "event_id": "evt_000001",
  "event_type": "TASK_COMPLETED",
  "node_id": "revision_compras",
  "actor": "usuario_77",
  "timestamp": "2026-06-08T16:05:00Z",
  "payload": {
    "decision": "approved",
    "comment": "Documentación validada"
  }
}
```

Parte A: Consultas que debe soportar el event log

En Cassandra el modelo se define a partir de las consultas. Primero se identifican las consultas reales sobre el event log y sus características.

Consulta	Usuario que la necesita	Frecuencia	Criticidad	Campos necesarios
Ver todos los eventos de una instancia	Operador, soporte	Alta	Alta	tenant_id, instance_id, timestamp, event_type, node_id, actor, payload

Consulta	Usuario que la necesita	Frecuencia	Criticidad	Campos necesarios
Ver eventos de un tenant en un rango de fechas	Auditor, administrador del tenant	Media	Alta	tenant_id, timestamp, instance_id, event_type, actor
Buscar errores técnicos recientes	Equipo de operaciones	Alta	Alta	tenant_id, timestamp, event_type = ERROR, instance_id, node_id, payload
Auditar acciones de un usuario	Auditor, seguridad	Baja	Media	tenant_id, actor, timestamp, event_type, instance_id
Calcular cantidad de eventos por proceso	Analista, dashboard	Media	Baja	tenant_id, process_id, contador
Reconstruir el estado de una instancia	Soporte, motor de reproceso	Media	Alta	tenant_id, instance_id, timestamp (orden), event_type, node_id, payload

Respuestas:

1. ¿Cuál es la consulta más importante para auditoría?
Ver eventos de un tenant en un rango de fechas. Es la base del informe de auditoría: permite revisar qué pasó en una ventana de tiempo sin depender de conocer instancias puntuales. La auditoría de acciones de un usuario la complementa.
2. ¿Cuál es la consulta más importante para operación diaria?
Ver todos los eventos de una instancia, junto con buscar errores técnicos recientes. El soporte las usa todo el día para diagnosticar instancias trabadas y detectar fallos en caliente.
3. ¿Cuál es la consulta más difícil de resolver eficientemente?
Auditar acciones de un usuario, porque el actor cruza instancias, procesos y tenants. Si la tabla está particionada por instancia o por fecha, esa consulta obligaría a un scan. Necesita su propia tabla particionada por actor.
4. ¿Qué consulta debería priorizarse en el primer prototipo del TPO?
Ver todos los eventos de una instancia (events_by_instance). Es la consulta de mayor frecuencia y criticidad, sostiene la reconstrucción de estado y modela el patrón base de event log en Cassandra (partición por instancia, orden por tiempo).

Parte B: Diseño orientado a consultas

No se parte de un modelo normalizado. Se define una tabla por patrón de consulta y se duplican los campos necesarios en cada una. El event log es append-only, así que la duplicación no genera problemas de actualización.

Tabla 1: events_by_instance

Elemento	Definición propuesta
Nombre conceptual de tabla	events_by_instance
Consulta que optimiza	Ver todos los eventos de una instancia y reconstruir su estado
Partition key	(tenant_id, instance_id)
Clustering key	event_time, event_id

Elemento	Definición propuesta
Orden esperado	event_time ascendente (cronológico, para replay del flujo)
Campos principales	process_id, node_id, event_type, actor, payload
Riesgo principal	Partición grande si una instancia genera muchos eventos. Acotado: una instancia BPM cierra y deja de crecer.

Tabla 2: events_by_tenant_date

Elemento	Definición propuesta
Nombre conceptual de tabla	events_by_tenant_date
Consulta que optimiza	Ver eventos de un tenant en un rango de fechas
Partition key	(tenant_id, date_bucket) con date_bucket = día (yyyy-mm-dd)
Clustering key	event_time, event_id
Orden esperado	event_time descendente (lo más reciente primero)
Campos principales	instance_id, process_id, event_type, node_id, actor
Riesgo principal	Hotspot del tenant dominante y partición grande si el bucket es muy amplio. Se mitiga con bucket diario (u horario si hace falta).

Tercera tabla (opcional), para cubrir el resto de las consultas de la Parte A.

Tabla 3: errors_by_tenant_date

Elemento	Definición propuesta
Nombre conceptual de tabla	errors_by_tenant_date
Consulta que optimiza	Buscar errores técnicos recientes de un tenant
Partition key	(tenant_id, date_bucket)
Clustering key	event_time, event_id
Orden esperado	event_time descendente
Campos principales	instance_id, process_id, node_id, payload
Riesgo principal	Partición chica (solo eventos ERROR), bajo riesgo. Si un tenant entra en falla masiva, el bucket diario acota el crecimiento.

Tabla 4: events_by_actor

Elemento	Definición propuesta
Nombre conceptual de tabla	events_by_actor
Consulta que optimiza	Auditar acciones de un usuario
Partition key	(tenant_id, actor)
Clustering key	event_time, event_id
Orden esperado	event_time descendente

Elemento	Definición propuesta
Campos principales	instance_id, process_id, event_type, node_id
Riesgo principal	Partición grande para un usuario muy activo. Se mitiga agregando un bucket temporal (tenant_id, actor, mes) si el volumen lo exige.

Para calcular cantidad de eventos por proceso se usa una tabla de contadores (events_count_by_process), separada porque mezcla un patrón de agregación con el log append-only.

Parte C: CQL conceptual

Versión conceptual de las tablas. Muestra claves y campos, no pretende ser sintaxis ejecutable afinada.

```
CREATE TABLE events_by_instance (
  tenant_id    text,
  instance_id  text,
  event_time   timestamp,
  event_id     text,
  process_id   text,
  node_id      text,
  event_type   text,
  actor        text,
  payload      text,
  PRIMARY KEY ((tenant_id, instance_id), event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time ASC);
```

```
CREATE TABLE events_by_tenant_date (
  tenant_id    text,
  date_bucket  date,
  event_time   timestamp,
  event_id     text,
  instance_id  text,
  process_id   text,
  node_id      text,
  event_type   text,
  actor        text,
  PRIMARY KEY ((tenant_id, date_bucket), event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time DESC);
```

```
CREATE TABLE errors_by_tenant_date (
  tenant_id    text,
  date_bucket  date,
  event_time   timestamp,
  event_id     text,
  instance_id  text,
  process_id   text,
  node_id      text,
  payload      text,
  PRIMARY KEY ((tenant_id, date_bucket), event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time DESC);
```

```
CREATE TABLE events_by_actor (
  tenant_id    text,
  actor        text,
  event_time   timestamp,
  event_id     text,
  instance_id  text,
  process_id   text,
  event_type   text,
  node_id      text,
  PRIMARY KEY ((tenant_id, actor), event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time DESC);
```

```
CREATE TABLE events_count_by_process (
  tenant_id    text,
  process_id   text,
```

```

    total        counter,
    PRIMARY KEY ((tenant_id, process_id))
);

```

El payload se modela como text (JSON serializado). Mantiene la flexibilidad del documento original sin obligar a un esquema rígido por tipo de evento.

Parte D: Análisis de particiones

Para cada tabla se analiza el riesgo de partición grande (una sola partición crece sin límite) y de hotspot (una partición concentra la carga).

Tabla	Riesgo de partición grande	Riesgo de hotspot	Causa	Mitigación
events_by_instance	Bajo a medio	Bajo	Una instancia con muchos eventos llena su partición. La carga se reparte porque cada instancia es una partición distinta.	La instancia cierra y deja de crecer. Si un proceso genera miles de eventos por instancia, sub-particionar con un bucket secuencial.
events_by_tenant_date	Medio	Alto	El tenant dominante concentra escrituras del día en una sola partición (tenant_id, día).	Bucket más fino (hora) para el tenant grande. Salting con un sufijo de bucket en la partition key.
errors_by_tenant_date	Bajo	Medio	Solo eventos ERROR, volumen chico salvo falla masiva de un tenant.	Bucket diario acota el tamaño. Bucket horario si hay tormenta de errores.
events_by_actor	Medio	Medio	Un usuario muy activo (o un actor de sistema) acumula muchos eventos en (tenant_id, actor).	Agregar bucket temporal (tenant_id, actor, mes) para repartir por período.

Preguntas guía:

1. ¿Qué pasa si una instancia genera miles de eventos?
En events_by_instance esa partición crece, pero queda acotada porque la instancia finaliza. Mientras siga abierta, las lecturas del historial completo leen una partición grande. Si el volumen por instancia es muy alto, se agrega un bucket secuencial a la partition key para partirla.
2. ¿Qué pasa si un tenant genera el 50% de todos los eventos?
La partición de ese tenant en events_by_tenant_date se vuelve un hotspot de escritura y de lectura. Particionar solo por tenant no alcanza: hay que sumar un bucket temporal fino o salting para repartir su carga entre varias particiones.
3. ¿Conviene agregar una fecha o bucket a la partition key?
Sí en las tablas por tenant. El bucket temporal (día, y hora si hace falta) evita que una partición por tenant crezca de forma indefinida y reparte la escritura en el tiempo. En events_by_instance no es necesario porque la instancia ya acota la partición.
4. ¿Conviene particionar por instance_id, por tenant_id, por fecha o por una combinación?
Por una combinación, según la consulta. Para el historial de una instancia, (tenant_id, instance_id). Para auditoría temporal, (tenant_id, date_bucket). No existe una única clave que sirva a todas las consultas: por eso se crea una tabla por patrón de acceso.

5. ¿Qué consulta se complica si se distribuye demasiado?

Reconstruir el estado completo de una instancia. Si sus eventos quedaran repartidos por fecha en muchas particiones, el replay sería multi-partición y lento. Por eso `events_by_instance` agrupa todos los eventos de la instancia en una sola partición, ordenados por tiempo.

Parte E: Consistencia configurable

Nivel de consistencia conceptual por operación sobre el event log. Se evalúa la coherencia entre criticidad, performance, disponibilidad y riesgo de inconsistencia. Se asume un factor de replicación $N = 3$.

Operación	Consistencia sugerida	Justificación
Insertar evento de auditoría	ONE	Escritura masiva append-only. ONE maximiza el throughput y la disponibilidad. La durabilidad la dan las tres réplicas, no el quórum, y los eventos no se sobrescriben.
Leer eventos de una instancia activa	QUORUM	Mientras la instancia está viva se opera sobre estos datos. QUORUM evita leer un historial stale al que le falten los últimos eventos.
Leer historial cerrado para auditoría	ONE	La instancia finalizó y el historial ya no cambia. Una réplica alcanza, porque todas convergieron. ONE da lectura rápida y barata.
Leer métricas agregadas	ONE	Dashboards que toleran datos de hace segundos. Se prioriza disponibilidad y latencia sobre exactitud al instante.
Reconstruir estado desde eventos	QUORUM	El replay alimenta decisiones (reproceso, soporte). Hay que leer todos los eventos confirmados, no una réplica que podría estar atrasada.

No hay una única respuesta correcta. El criterio es elegir consistencia fuerte (QUORUM) donde el dato alimenta una decisión sobre una instancia viva, y consistencia eventual (ONE) donde el dato es inmutable o tolera atraso.

Parte F: Comparación con MongoDB

Criterio	MongoDB para eventos	Cassandra para eventos	Decisión del grupo
Escritura masiva	Buena, pero el modelo de réplica con primario único limita el throughput de escritura sostenida.	Escritura distribuida sin maestro: cada nodo acepta escrituras. Pensado para append masivo.	Cassandra
Consulta por instancia	Directa con índice por (tenant_id, instance_id).	Directa con partition key (tenant_id, instance_id), lectura secuencial y ordenada.	Empate, leve ventaja Cassandra por orden físico

Criterio	MongoDB para eventos	Cassandra para eventos	Decisión del grupo
Consulta por rango temporal	Posible con índice sobre timestamp, pero compite con la carga de escritura.	Natural con clustering key temporal y bucket por fecha.	Cassandra
Flexibilidad del payload	Alta: documento embebido, consultable campo a campo.	Media: payload como JSON en un campo text, no se consulta por subcampo.	MongoDB
Escalabilidad horizontal	Sharding disponible, con más complejidad operativa.	Escalabilidad horizontal lineal por diseño, agregar nodos es la operación normal.	Cassandra
Simplicidad del prototipo	Alta: una sola base para definiciones, instancias y un primer event log.	Media: obliga a pensar el modelo por consultas y varias tablas desde el inicio.	MongoDB

Respuestas:

1. ¿Usarían MongoDB para el event log en el prototipo inicial?
Sí, para el primer prototipo. MongoDB ya almacena definiciones e instancias (Práctica 5). Un event log inicial de bajo volumen puede vivir en una colección, sin sumar otra tecnología antes de tiempo.
2. ¿Usarían Cassandra para el diseño final esperado?
Sí. Con el volumen proyectado (millones de eventos por día, append-only, crecimiento indefinido), el patrón es exactamente el de Cassandra: escritura distribuida, particionado por tenant e instancia, orden temporal.
3. ¿Qué se gana y qué se pierde en cada alternativa?
Con MongoDB se gana simplicidad y flexibilidad del payload, se pierde throughput de escritura y escalabilidad horizontal a gran volumen. Con Cassandra se gana escritura masiva y escalabilidad lineal, se pierde flexibilidad de consulta sobre el payload y se paga un modelo más rígido, con una tabla por consulta.
4. ¿Qué decisión defenderían en la presentación del TPO?
Arquitectura polígloa por etapas. MongoDB como fuente de definiciones e instancias y event log inicial. Cassandra como destino del event log cuando el volumen y el pico de escritura lo justifiquen. Las claves se diseñan desde ya (tenant_id, instance_id, timestamp) para migrar sin rediseñar.